

PHYS 4027 / 5027 • FALL 2026 • LECTURE 8

Adaptive Step Size Control



Prof. Alex Chen

Department of Physics

Washington University in St. Louis

Today

- **Estimating the error of a step.** Step doubling: take each step twice.
- **From error to step size.** One number decides accept, reject, and the next h .
- **A worked example.** The integrator discovers the timescale on its own.
- **Embedded methods.** The error estimate for free: Dormand-Prince and Cash-Karp.

Part I · Estimating the error



What one step can learn about itself

Last time

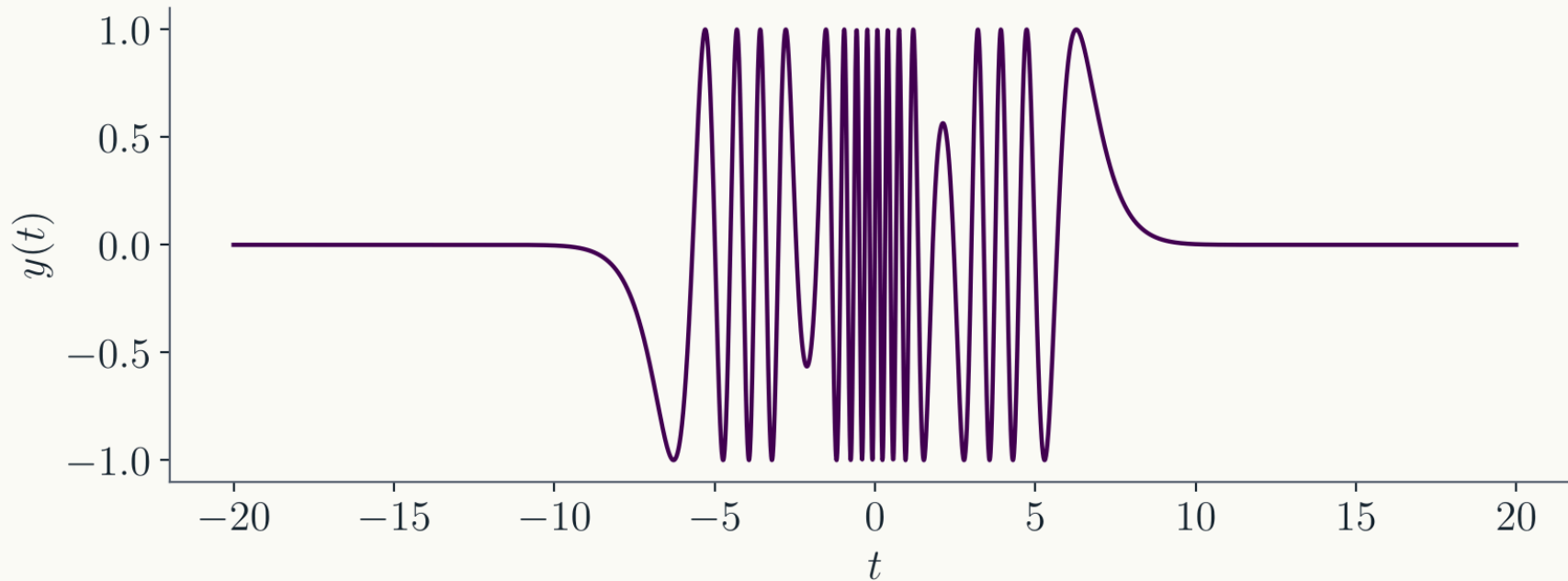


The Runge-Kutta family: estimate a few slopes, step with a weighted average.

$$k_i = f\left(t_n + c_i h, y_n + h \sum_{j=1}^{i-1} a_{ij} k_j\right), \quad y_{n+1} = y_n + h \sum_{i=1}^s b_i k_i$$

- In everything so far, h is fixed: chosen beforehand, the same for every step.
- Chosen how? Accuracy and stability both constrain it — but you have to guess before you have seen the solution.

The problem with a fixed step



$$y(t) = \sin \left(e^{-t^2/a^2} \omega t \right), \text{ with } a = 3, \omega = 20$$

- What fixed h would you pick for $t \in [-20, 20]$?
- The h that resolves the burst at $t = 0$ is absurdly small for the flat wings. Whatever you choose is wrong somewhere.

Step doubling



To adapt h we first need to know the error of a step. Simplest idea: take the step twice.

$$\begin{aligned}y^{(1)} &= \text{RK4}(t_n, y_n, h) \\y^{(2)} &= \text{RK4}\left(t_n + \frac{h}{2}, \text{RK4}\left(t_n, y_n, \frac{h}{2}\right), \frac{h}{2}\right)\end{aligned}$$

- One full RK4 step of h , and two RK4 steps of $h/2$, from the same starting point.
- The error estimate is the difference: $\varepsilon = |y^{(1)} - y^{(2)}|$.
- How many evaluations of f does one trial cost?

How the estimate scales with h



RK4 is 4th-order accurate, so one step of size h carries a local error $c h^5$:

$$y(t_n + h) = y^{(1)} + c h^5 + O(h^6)$$

$$y(t_n + h) = y^{(2)} + 2 c (h/2)^5 + O(h^6)$$

- Subtracting: $\varepsilon = |y^{(1)} - y^{(2)}| = \frac{15}{16} c h^5 \propto h^5$.
- The half-step result is 16 times better; its true error is roughly $\varepsilon/15$.

Part II · From error to step size



Choosing h according to estimated error

The control law



Suppose the step just taken gave error ε , and we want the next one to give $\varepsilon_{\text{target}}$. Since $\varepsilon = Ch^5$ with the same C nearby:

$$\frac{\varepsilon_{\text{target}}}{h'^5} = \frac{\varepsilon}{h^5} \implies h' = h \left(\frac{\varepsilon_{\text{target}}}{\varepsilon} \right)^{1/5}$$

The same formula works in both directions: it shrinks h when the step was too coarse, and grows h when the step was wastefully accurate.

Choosing the target error



We know nothing about $y(t)$ in advance, so the standard target combines two tolerances:

$$\epsilon_{\text{target}} = \epsilon_{\text{abs}} + \epsilon_{\text{rel}} |y|$$

- Pure relative fails when y passes through zero: the target collapses and no step is small enough.
- Pure absolute fails when you do not know the scale of y in advance.
- Which y goes into the target, $y^{(1)}$ or $y^{(2)}$? A free choice; they differ only at the order we are estimating.

Systems: one h , many errors



For N_{eq} coupled equations, every component y_i has its own ε_i and target — but there is only one h . Collapse them into a scalar:

$$\text{err} = \sqrt{\frac{1}{N_{\text{eq}}} \sum_{i=0}^{N_{\text{eq}}-1} \left(\frac{\varepsilon_i}{\varepsilon_{\text{target},i}} \right)^2}$$

- Each term is the error of one component in units of its own target.
- $\text{err} < 1$ means every component is (on RMS average) within tolerance: the step is good.

The safety factor



In terms of the scalar error, the update becomes

$$h' = S h \left(\frac{1}{\text{err}} \right)^{1/5}$$

- S is a *safety factor*, slightly below 1; in practice $S = 0.9$ works well.
- Why not $S = 1$? The h^5 law is only the leading term. Aiming exactly at $\text{err} = 1$ means half the steps land just above it.

A rejected step throws away 11 evaluations of f . Undershooting the step size by 10% is far cheaper than rejecting.

The full algorithm

1. From (t, y) , compute $y^{(1)}$ (one step of h) and $y^{(2)}$ (two steps of $h/2$).
2. For each component, form $\varepsilon_i = |y_i^{(1)} - y_i^{(2)}|$ and $\varepsilon_{\text{target},i} = \epsilon_{\text{abs}} + \epsilon_{\text{rel}} |y_i|$.
3. Combine into the scalar err (previous slide).
4. If $\text{err} < 1$: accept, advance $t \rightarrow t + h$ with $y^{(2)}$. Otherwise: reject, stay at t .
5. Either way, set $h' = S h (\text{err})^{-1/5}$ and continue. After an accepted step this usually *grows* h .

Rejections are not failures — they are the controller braking when the solution speeds up.

Practical refinements



- Do not grow h on the step right after a rejection; and cap the growth (e.g. at $5\times$) so one flat stretch cannot fling h to infinity.
- An empirically more stable update keeps some memory of the previous step:

$$h' = S h (\text{err})^{-0.7/5} (\text{err}_{\text{prev}})^{0.4/5}$$

- Use an *embedded* method to cut the 11 evaluations per trial (Part IV).
- Use interpolation to produce output at regular t values even though the steps are irregular (*dense output* — next lecture).

Part III · The example



The integrator finds the timescale

Setting up the run

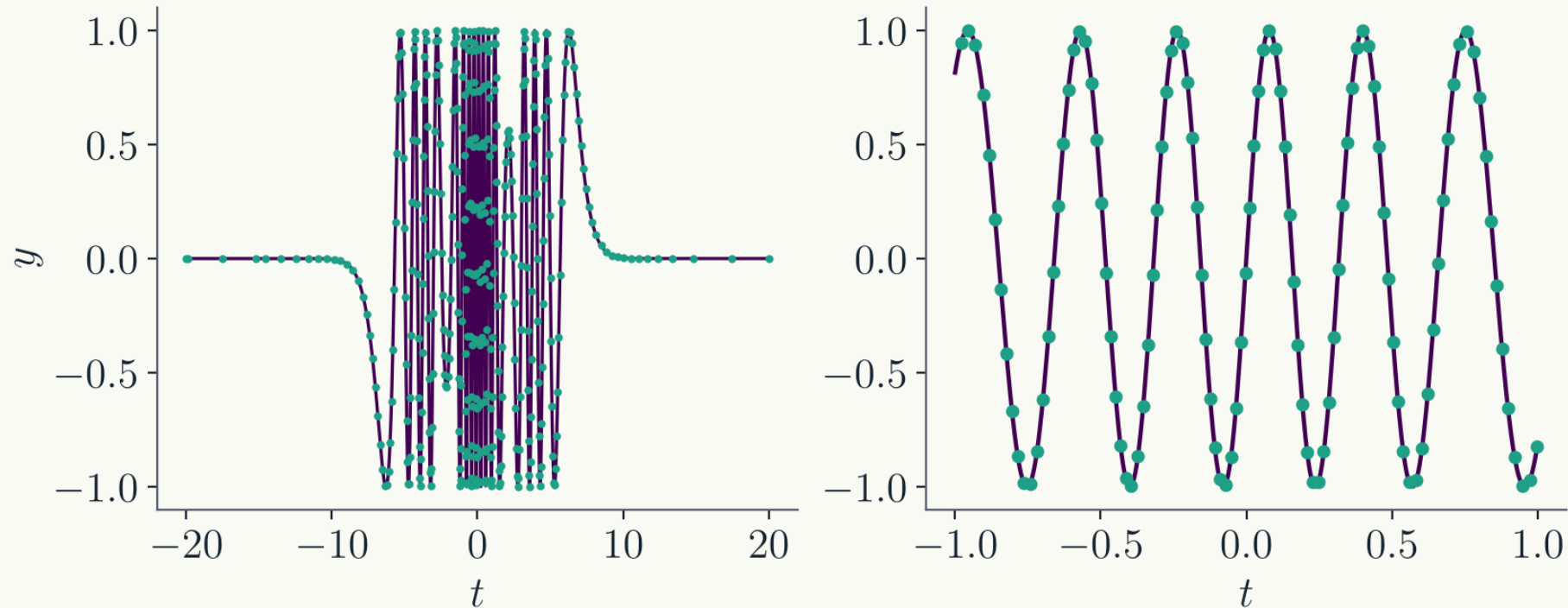


The chirped pulse from Part I is the solution of

$$\frac{dy}{dt} = \omega e^{-t^2/a^2} \left(1 - \frac{2t^2}{a^2} \right) \cos \left(e^{-t^2/a^2} \omega t \right)$$

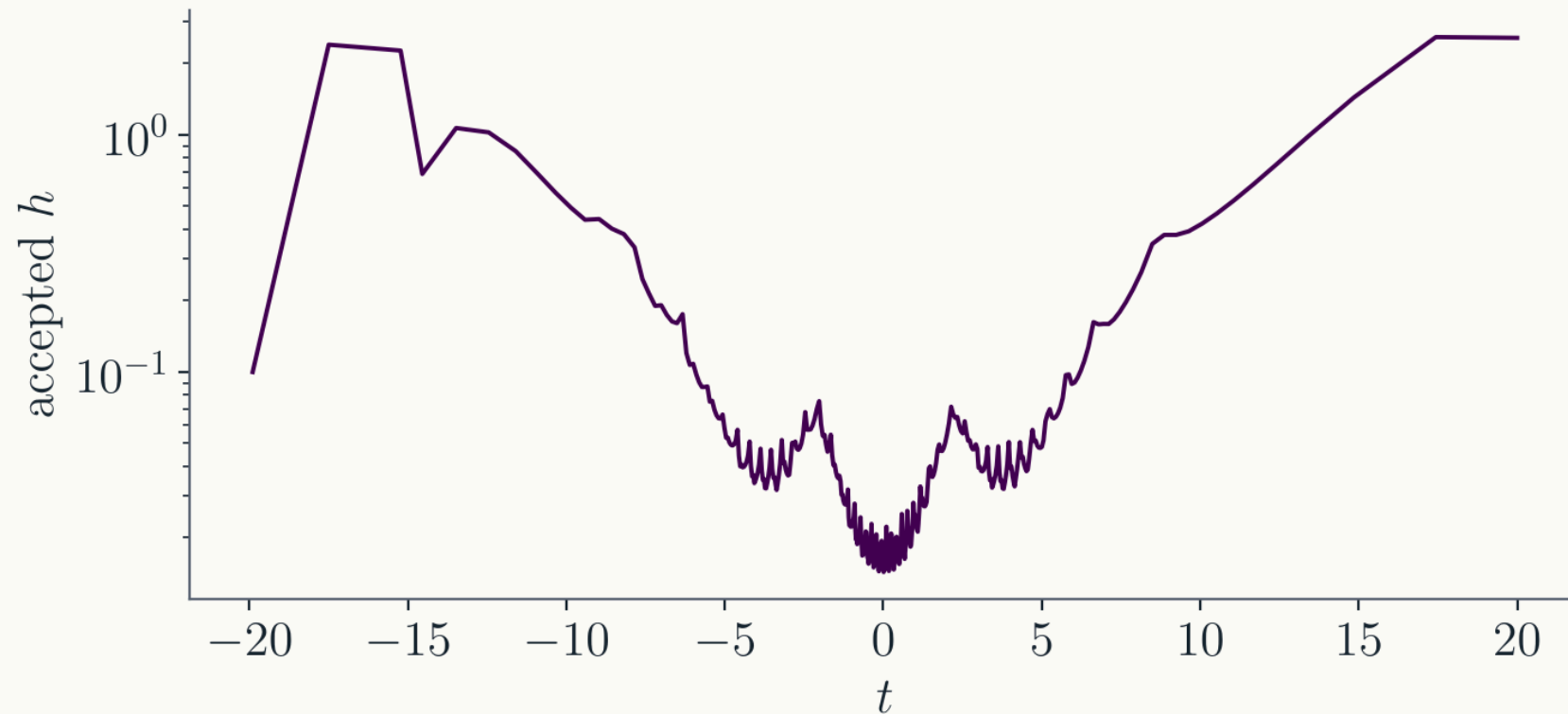
- Adaptive RK4 by step doubling, exactly the algorithm of Part II.
- $\epsilon_{\text{abs}} = \epsilon_{\text{rel}} = 10^{-6}$, $S = 0.9$, initial $h = 0.1$, from $t = -20$ to 20 .

The computed solution



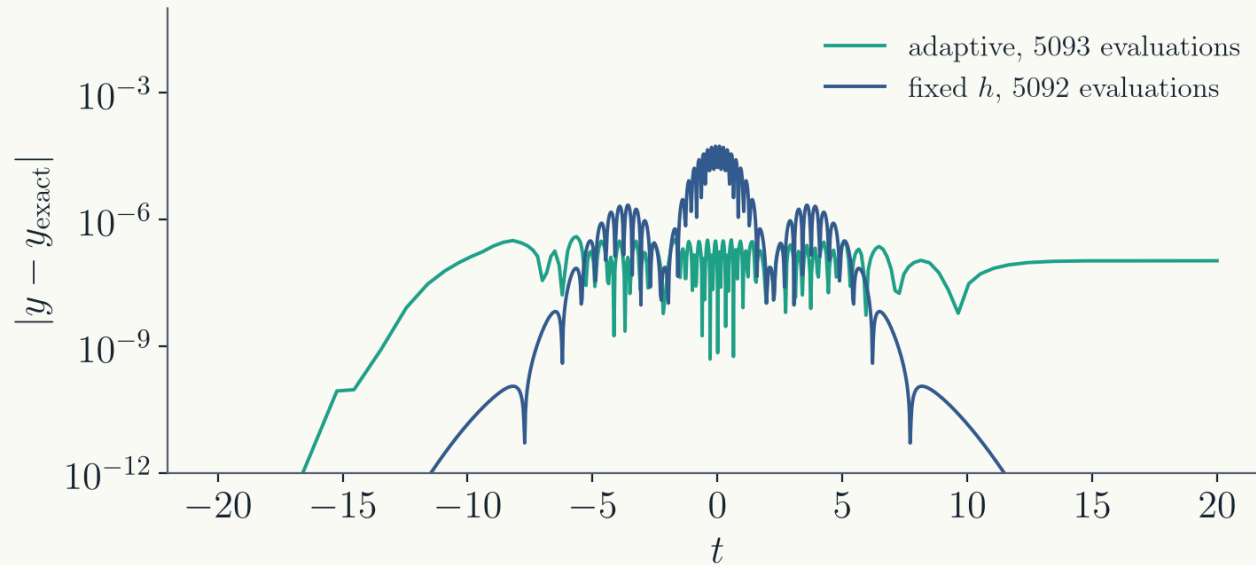
- Every accepted step is a dot, drawn on the analytic solution. Right: a zoom into the center of the burst.
- The dots land on the curve everywhere — and they crowd into the burst and spread out over the wings.

The step size finds the physics



- 378 accepted steps, 85 rejected, 5093 evaluations of f . The accepted h spans 0.014 to 2.6 — a factor of 180.
- Nobody told the integrator where the burst is. The error estimate discovered it.

Was it worth it?



- Fixed-step RK4 with the *same* 5093 evaluations: max error 5×10^{-5} , versus 4×10^{-7} adaptive — 140 times worse.
- To match the adaptive error it needs about 19,400 evaluations: 4 times the cost.

Fixed h wastes accuracy in the wings (error 10^{-10} where 10^{-6} would do) and still blows the budget in the burst. Adaptivity spends the error budget uniformly.

Part IV · Embedded methods



An error estimate for free

The cost of knowing the error

- Step doubling pays 11 evaluations per trial, and incur more evaluations if the step is rejected. The example costed 13.5 evaluations per accepted step against 4 for plain RK4.
- The estimate came from comparing two methods of *different accuracy* on the same step. But nothing says the two methods must be built from different evaluations.
- *Embedded methods*: two weighted averages of the **same** stages k_i , one of order p and one of order $p - 1$. The error estimate then costs nothing extra.

An embedded Butcher tableau

An embedded tableau carries two bottom rows: the step is taken with b_i , and the second row b_i^* gives a lower-order companion.

c_1					
c_2	a_{21}				
c_3	a_{31}	a_{32}			
$\vdots$	$\vdots$	$\vdots$	$\ddots$		
c_s	a_{s1}	a_{s2}	$\cdots$	$a_{s,s-1}$	
	b_1	b_2	$\cdots$	b_{s-1}	b_s
	b_1^*	b_2^*	$\cdots$	b_{s-1}^*	b_s^*

$$\varepsilon = |y_{n+1} - y_{n+1}^*| = h \left| \sum_{i=1}^s (b_i - b_i^*) k_i \right|$$

Dormand-Prince 5(4)

Last lecture's Dormand-Prince tableau, now with its second row: a 5th-order method with an embedded 4th-order estimate.

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	-56/15	32/9				
8/9	19372/6561	-25360/2187	64448/6561	-212/729			
1	9017/3168	-355/33	46732/5247	49/176	-5103/18656		
1	35/384	0	500/1113	125/192	-2187/6784	11/84	
	35/384	0	500/1113	125/192	-2187/6784	11/84	0
	5179/57600	0	7571/16695	393/640	-92097/339200	187/2100	1/40

- Seven stages, but the last stage row equals the b row: k_7 of this step is k_1 of the next, a property called FSAL ("first same as last"). Six new evaluations per accepted step, error estimate included.
- This is the default method behind `solve_ivp` in `scipy`.

Cash-Karp 5(4)

Another embedded 5(4) pair, with friendlier coefficients:

0						
1/5	1/5					
3/10	3/40	9/40				
3/5	3/10	-9/10	6/5			
1	-11/54	5/2	-70/27	35/27		
7/8	1631/55296	175/512	575/13824	44275/110592	253/4096	
	37/378	0	250/621	125/594	0	512/1771
	2825/27648	0	18575/48384	13525/55296	277/14336	1/4

- Six stages, no FSAL: all six evaluations are fresh each step. Developed with semi-stiff problems in mind; it behaves slightly better than Dormand-Prince there.

Using an embedded method



Only step 1 of the algorithm changes:

~~Compute $y^{(1)}$ and $y^{(2)}$ using one RK4 step of h and two of $h/2$:~~

Compute $y^{(1)}$ and $y^{(2)}$ from the two bottom rows of the Dormand-Prince tableau.

- Always advance with the 5th-order $y^{(1)}$; the 4th-order $y^{(2)}$ exists only to be subtracted.
- Everything else — targets, the scalar err, accept/reject, $h' = S h (\text{err})^{-1/5}$ — carries over unchanged.

Adaptive DP45, complete



1. From (t, y) , compute the seven stages $k_1, \dots, k_7$; form $y^{(1)}$ (row b) and $y^{(2)}$ (row b^*).
2. $\epsilon_i = |y_i^{(1)} - y_i^{(2)}|$, $\epsilon_{\text{target},i} = \epsilon_{\text{abs}} + \epsilon_{\text{rel}} |y_i^{(1)}|$.
3. Combine into the scalar err.
4. $\text{err} < 1$: accept, advance with $y^{(1)}$, reuse k_7 as the next k_1 . Otherwise reject. Either way $h' = S h (\text{err})^{-1/5}$.

This loop, plus the refinements of Part II, is what production ODE solvers actually run.

Where this goes next

- **Next lecture:** stopping at a condition rather than a time, dense output, and parallel programming. Everything you need for Project 1.
- References:
 - Numerical Recipes, 3rd ed., Section 17.2.
 - Hairer, Nørsett & Wanner, *Solving Ordinary Differential Equations I*, Section II.4.